The Journal of Open Source Software

SciWIn-Client: Reproducible Computational Workflows from the Command-line

Jens Krumsieck [1], **Antonia Leidel** [2], **Xaver Stiensmeier** [3], **Patrick König** [2], **Florian Hoedt** [1,4], **and Harald von Waldow** [1]

1 Johann Heinrich von Thünen Institute, Braunschweig, Germany **2** Leibniz Institute of Plant Genetics and Crop Plant Research, Gatersleben, Germany **3** Bielefeld University, Bielefeld, Germany **4** PowerCo SE, Salzgitter, Germany (current affiliation)

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

SciWIn-Client is a command-line tool developed within the FAIRagro consortium (Ewert et al., 2023) as part of the Scientific Workflow Infrastructure (SciWIn). It is designed to streamline the creation, execution and management of reproducible computational workflows using the *Common Workflow Language (CWL)*(Crusoe et al., 2022). By wrapping ordinary command-line commands with a thin layer of tooling, SciWIn-Client automatically generates CWL definitions, allowing scientists to write CWL using the well-known commands rather than hand-authoring verbose specifications. Implemented in Rust for reliability, SciWIn-Client integrates natively with Git for version control and provenance tracking. It supports both local and remote workflow execution and is interoperable with the Workflow RO-Crate (Bacall et al., n.d.) and Workflow Run RO-Crate (Leo et al., 2024) standards. Furthermore SciWIn-Client is interoperable with research data management frameworks such as DataPLANT's ARC format (DataPLANT, 2025; Weil et al., 2023).

Statement of need

Automated computational workflows are essential for managing complex, multi-step data analysis across various scientific disciplines. Significant effort has been invested into domain-specific languages that formalize and standardize computational scientific processes, thereby enhancing reproducibility, scalability and efficiency. To harmonize those workflow languages, the Common Workflow Language (CWL) was introduced as universal standard (Crusoe et al., 2022). Its design emphasizes flexibility and machine readability but its verbose YAML-based syntax poses a substantial barrier to adoption among researchers unfamiliar with structured data formats.

CWL therefore is predestined to be written by machines rather than humans, which ultimately motivated the conception of SciWIn-Client. SciWIn-Client provides an intuitive command-line interface that automates CWL generation and management. It translates typical research computing tasks into structured, version-controlled workflow definitions, effectively allowing scientists to "write CWL by doing science."

State of the field

Tools such as Nextflow (Di Tommaso et al., 2017), Snakemake (Mölder et al., 2021) and Galaxy (Giardine et al., 2005; The Galaxy Community, 2024) have achieved significant adoption within the scientific community. They offer powerful execution environments and rich graphical or scripting environments. Significant effort was put in providing a broad set of workflows

DRAFT

especially for the OMICS-community (e.g. nf-core (Ewels et al., 2020)), but they are lacking in the agro-community where individual scripting plays a key part.

Bringing such individual scripts into established workflow platforms can present a substantial hurdle. With Nextflow, researchers need to learn the Groovy-based workflow DSL. In Galaxy, tools may need to undergo a curation and integration process before they become available on a public usegalaxy.* server. Moreover, workflows authored for Galaxy are often coupled to the capabilities and configuration of a particular Galaxy instance, and portability across infrastructures may require additional adaptation.

CWL was introduced as a vendor-neutral, platform agnostic standard to address fragmentation and portability. There were efforts to make Galaxy and Nextflow compliant to this standard (Di Tommaso & Sayers, 2018). However, the cwl2nxf converter for Nextflow was archived in 2025, and CWL support in Galaxy has been an open draft since 2021 (Soranzo, 2021). Snakemake has `--export-cwl` which however just wraps a call to the Snakemake program in a CWL description using the Snakemake Docker container. The adoption of CWL remains lower than that of Snakemake, Nextflow and Galaxy. Its verbose, YAML-based syntax requires familiarity with structured data formats and workflow abstractions that many domain researchers do not possess. The CWL ecosystem further compounds this problem. While a number of number of great runner implementations exist (e.g. cwltool, Toil (Vivian et al., 2017), REANA, Arvados)(Common Workflow Language, n.d.-b), they require the Windows Subsystem for Linux (WSL) on Windows. Moreover, the space of authoring tools is sparse (Common Workflow Language, n.d.-a). Rabix offered a graphical editor (Rabix Composer)(Kaushik et al., 2016) but further development was made closed-source and moved into the Seven Bridges Platform (Rabix Composer contributors, 2021). The open-sourced version has been unmaintained for over 5 years and is significantly outdated. Many other generators like `scriptcwl` (Zwaan et al., 2019) (last commit 2020) are outdated as well or restricted to Python, Python argparse programs or webforms like `Janis` (PMCC Bioinformatics Core, 2023) (last commit 2023), `argparse2tool`(Rasche & Khodak, 2024) (last commit 2024) or `ToolJig` (Piccolo et al., 2021). The `zatsu-cwl-generator` (Tanjo & Ishii, 2022) (last commit 2022) is the closest to our approach, but it parses a command without running it, so it cannot observe files the command creates.

SciWIn-Client addresses this authoring gap by allowing researchers to create CWL workflows from their existing command-line-oriented workflows without requiring them to author the CWL document manually. It has no dependency on a specific workflow platform, integrates with Git-based workflows, and is distributed as a single self-contained binary.

A conceptually close work are tools capturing the computational environment or provenance through execution, e.g. ReproZip (Chirigati et al., 2016), noWorkflow (Pimentel et al., 2017), and Sciunit (Ton That et al., 2017). They demonstrate that execution traces and runtime information can be used to support reproducibility and provenance capture. However, their primary objective is to capture, package, or reproduce information about an execution, whereas SciWIn-Client builds on this information to generate a machine-actionable workflow description.

Software design

SciWIn-Client (short: `s4n`) was built around one simple design decision. A CWL `CommandLineTool` document is created from observing a command's actual execution, rather than by parsing the command line or asking the user to fill in a form. A command line alone provides no information about which files are written, so a purely static approach (such as `zatsu-cwl-generator`) cannot produce the `outputs` of a tool. Everything else follows from this decision and the goal of seamlessly integrating into the researcher's existing mode of operation: using a terminal, scripts and Git. Git integration also provides built-in version control and interoperability with research data management frameworks such as DataPLANT's ARC (DataPLANT, 2025; Weil et al., 2023) format which can be viewed as a Git-based

DRAFT

implementation of the RO-Crate standard(Soiland-Reyes et al., 2022).

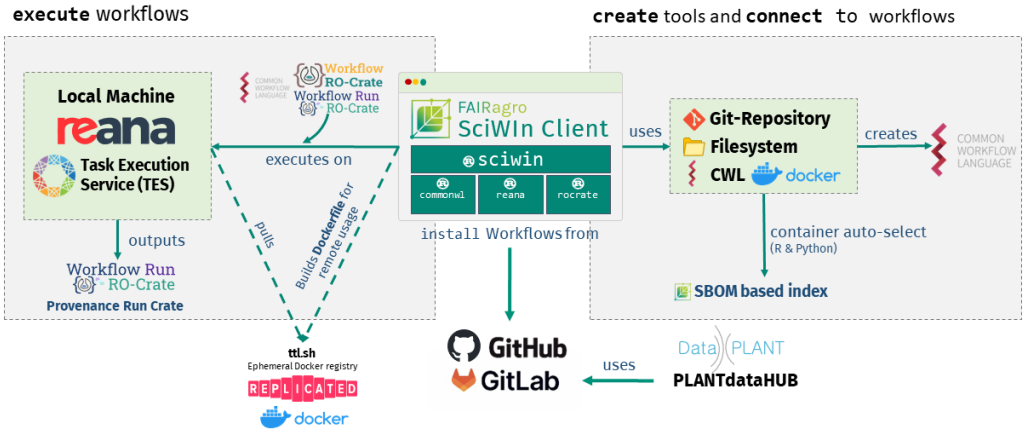


Figure 1: Overview of SciWIn-Client design. The `s4n` binary is a thin front end over the shared `sciwin` library, which builds on the `commonwl`, `reana` and `rocrate` libraries. Tools are authored by observing a command inside a Git repository, are connected into workflows, and are executed on a local, Docker, TES or REANA backend. Each run can be exported as a Workflow Run RO-Crate.

Architecture

`s4n` is a thin command-line layer around the `sciwin` library, so that multiple frontends can be supported. `sciwin` is organized into authoring (tool and workflow generation), execution, provenance, container resolution and repository handling. It combines three further libraries that were developed alongside it because no promising equivalent existed in the Rust ecosystem: `commonwl` (CWL parsing and an execution engine), `reana` (a client for the REANA (Šimko et al., 2019) API) and `rocrate` (reading and building RO-Crates). They are published on `crates.io` separately, so they can be reused independently of SciWIn.

Rust was chosen for practical reasons beyond type safety. It produces a single self-contained binary without a language runtime, which lowers the installation and configuration barrier for researchers, and it runs natively on Windows, Linux and macOS. This fills a gap the reference CWL runner has, as it requires WSL on Windows. The price is a smaller pool of potential scientific contributors compared to Python, and the need to implement a CWL engine instead of reusing an existing one.

An overview of the architecture is shown in Figure 1.

Authoring tools

When users invoke a command or script using the `s4n create` prefix, SciWIn-Client automatically generates a tool following a simple 4 step process:

1. The command-line is used to parse `inputs` and `baseCommand` to construct a preliminary `CommandLineTool`.
2. The preliminary `CommandLineTool` is executed locally ("probe"). Git is used in background to determine changes in the file system. Changed files and directories are added as `outputs`.
3. Requirement metadata such as containers, network access, environment variables are added to the preliminary `CommandLineTool` by `s4n create`'s command options.
4. The preliminary `CommandLineTool` is finalized by post processing steps such as EDAM format annotation, path relativization and array input detection.

Example:

DRAFT

```
s4n create -c Dockerfile --container-tag pyplot --enable-network \
python3 code/plot_election.py --data data.csv --features features.json
```

would produce a `CommandLineTool` with two prefixed inputs as well as Docker- and Network-Requirements:

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.2
class: CommandLineTool

requirements:
- class: InitialWorkDirRequirement
  listing:
  - entryname: code/plot_election.py
    entry:
      $include: ../../code/plot_election.py
- class: DockerRequirement
  dockerFile:
    $include: ../../Dockerfile
  dockerImageId: pyplot
- class: NetworkAccess
  networkAccess: true

inputs:
- id: data
  type: File
  default:
    class: File
    location: ../../data.csv
  inputBinding:
    prefix: --data
- id: features
  type: File
  default:
    class: File
    location: ../../features.json
  inputBinding:
    prefix: --features

outputs:
- id: election
  type: File
  outputBinding:
    glob: election.png

baseCommand:
- python
- code/plot_election.py
```

As Git is used to determine changes, a Git repository is a hard requirement. However, version control is already best practice for the intended users and it provides versioning of the generated files for free. A `.gitignore` can be used to ignore temporary directories or files may be created by the command.

While the system can automatically infer `inputs` and `outputs`, it also has inherent limits. Therefore users also have the option to define `inputs` and `outputs` explicitly.

DRAFT

Containers

A tool recorded from a local run depends on the researcher's environment. To make the resulting CWL portable, a container can be attached either as an image reference or as a path to a local Dockerfile that is built on execution. For Python and R scripts, SciWIn-Client is able to resolve a container automatically from the project's dependency files (`requirements.txt`, `pyproject.toml`, `DESCRIPTION`) using a curated SBOM-based registry maintained by the project.

Connecting tools

`s4n connect` links the output of one tool to the input of another by name or adds new workflow-wide `inputs` and `outputs`. The workflow is therefore built incrementally from tools that already exist, and the generated CWL Workflow stays a plain, editable document.

Workflow Execution

SciWIn-Client supports workflow execution on multiple backends (see Table 1) through the `s4n execute` command. The desired backend can be selected using the `--engine` flag. `s4n execute` accepts a CWL document or a Workflow RO-Crate (Bacall et al., n.d.) or Workflow Run RO-Crate (Leo et al., 2024) for every backend.

Table 1: Overview of available backends.

Backend	Purpose
`local`	Runs on the researcher's machine. Steps with a `DockerRequirement` run in Docker, Singularity or Podman.
`docker`	Runs every step in a container through the local Docker daemon.
`tes`	Submits tasks to a GA4GH Task Execution Service (TES) 1.1 server (Kanitz et al., 2024).
`reana`	Submits the workflow to a REANA instance. (Not part of `commonwl`)

The conformance with the CWL conformance test suite of `commonwl`'s backends is tracked using continuous integration and is 99% (97% required) across all current backends.

Trade-offs and limitations

- A Git repository is required, and observation only captures effects visible in the working tree.
- Authored tools are only as portable as their container definition. Without a container, a tool depends on the local environment.
- The tool targets command-line driven workflows. Interactive or graphical programs cannot be recorded.

Research impact statement

SciWIn-Client was used within FAIRagro to build the workflow for Use Case 6 (high-throughput crop growth simulation) (Gitahi et al., 2026; Gitahi, 2026). The resulting workflow was published on WorkflowHub (Gitahi, 2026; Gustafsson et al., 2025) and annotated using

DRAFT

DataPLANT tooling, which shows that SciWIn-created workflows can be moved into ARC-based research data management (Brilhaus et al., 2026). Workflows can be executed on the FAIRagro REANA instance operated in the de.NBI Cloud.

The software has been taught and presented to its target community of agricultural and life science researchers. SciWIn-Client was presented at the 2nd Conference on Research Data Infrastructure (CoRDI 2025) (Krumsieck, Leidel, König, et al., 2025a, 2025b). A hands-on workshop on SciWIn-Client was held at the Boosting Biodata Bootcamp 2026 (Leidel et al., 2026). It has also been demonstrated at several further events, including the NFDI4LS Conference and the FAIRagro Community Summit (Krumsieck, Leidel, Stiensmeier, et al., 2025, 2026a, 2026b; Krumsieck, 2025).

The release binaries have been downloaded more than 1700 times as of October 2026 (Collected download data). The supporting libraries `commonwl`, `reana` and `rocrate` are published separately on `crates.io`.

The source code is openly available at https://github.com/fairagro/sciwin under the MIT or Apache-2.0 license, and the project welcomes community contributions.

CRediT authorship contribution statement

Jens Krumsieck: Conceptualization, Methodology, Software, Validation, Writing - Original Draft, Writing - Review & Editing, Visualization, **Antonia Leidel**: Conceptualization, Methodology, Software, Validation, Writing - Original Draft, Writing - Review & Editing, Visualization, **Xaver Stiensmeier**: Conceptualization, Methodology, Validation, Writing - Original Draft, Writing - Review & Editing, **Patrick König**: Conceptualization, Methodology, Project administration, **Florian Hoedt**: Conceptualization, Funding acquisition, **Harald von Waldow**: Conceptualization, Methodology, Writing - Original Draft, Writing - Review & Editing, Supervision, Project administration

Acknowledgements

We gratefully acknowledge the financial support of the German Research Foundation (DFG) – project number 501899475.

AI usage disclosure

All paper content was written manually and reflects the careful thought and input of the authors. SciWIn is an open source project, and as such contributors are free to use any tools, AI or otherwise, to generate code contained in pull requests or commits. Claude Code was used as assistant to help with implementation of individual parts. It was mainly used for code generation, refactoring and documentation. All commits and pull requests are reviewed by the core developers and often iterated on multiple times; therefore, all content in the repository represents the effort and judgment of the authors.

References

Bacall, F., Williams, A. R., Owen, S., Soiland-Reyes, S., Chadwick, E., & Leo, S. (n.d.). *Workflow RO-crate profile 1.1*. WorkflowHub. Retrieved September 21, 2026, from https://w3id.org/workflowhub/workflow-ro-crate/1.1

Brilhaus, D., Jung, J., Krumsieck, J., Leidel, A., Martini, D., Mühlhaus, T., Ott, C., Perzlmaier, L., Reinosch, N., Sapir, P., Stiensmeier, X., Waldow, H. von, & Weil, H. L. (2026). *DataPLANT / FAIRagro hackathon may 4 - 5 2026 in darmstadt: Integrating ARCs and*

DRAFT

SciWIn: Tackling large data files, workflow validation, and use case ARCification. Zenodo. https://doi.org/10.5281/zenodo.22661911

Chirigati, F., Rampin, R., Shasha, D., & Freire, J. (2016). ReproZip. In F. Özcan, G. Koutrika, & S. Madden (Eds.), *Proceedings of the 2016 international conference on management of data* (pp. 2085–2088). ACM. https://doi.org/10.1145/2882903.2899401

Common Workflow Language. (n.d.-a). *Development tools*. Retrieved September 21, 2026, from https://www.commonwl.org/tools/

Common Workflow Language. (n.d.-b). *Implementations*. Retrieved September 21, 2026, from https://www.commonwl.org/implementations/

Crusoe, M. R., Abeln, S., Iosup, A., Amstutz, P., Chilton, J., Tijanić, N., Ménager, H., Soiland-Reyes, S., Gavrilović, B., Goble, C., & Community, T. C. (2022). Methods included. *0001-0782*, *65*(6), 54–63. https://doi.org/10.1145/3486897

DataPLANT. (2025). *nfdi4plants/ARC-specification: Annotated research context specification v3.0-draft.2*. Zenodo; Zenodo. https://doi.org/10.5281/ZENODO.8302661

Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, *35*(4), 316–319. https://doi.org/10.1038/nbt.3820

Di Tommaso, P., & Sayers, K. (2018). *cwl2nxf*. https://github.com/nextflow-io/cwl2nxf

Ewels, P. A., Peltzer, A., Fillinger, S., Patel, H., Alneberg, J., Wilm, A., Garcia, M. U., Di Tommaso, P., & Nahnsen, S. (2020). The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology*, *38*(3), 276–278. https://doi.org/10.1038/s41587-020-0439-x

Ewert, F., Specka, X., Anderson, J. M., Arend, D., Asseng, S., Boehm, F., Feike, T., Fluck, J., Gackstetter, D., Gonzales-Mellado, A., Hartmann, T., Haunert, J.-H., Hoedt, F., Hoffmann, C., König, P., Lesch, S., Lindstädt, B., Lischeid, G., Martini, D., … Weiland, C. (2023). *FAIRagro - a FAIR data infrastructure for agrosystems (proposal)*. Zenodo. https://doi.org/10.5281/ZENODO.8366884

Giardine, B., Riemer, C., Hardison, R. C., Burhans, R., Elnitski, L., Shah, P., Zhang, Y., Blankenberg, D., Albert, I., Taylor, J., Miller, W., Kent, W. J., & Nekrutenko, A. (2005). Galaxy: A platform for interactive large-scale genome analysis. *Genome Research*, *15*(10), 1451–1455. https://doi.org/10.1101/gr.4086505

Gitahi, J. (2026). *UC6 - automated data flows for crop simulation models*. WorkflowHub. https://doi.org/10.48546/WORKFLOWHUB.WORKFLOW.2236.1

Gitahi, J., Leroy, B. M., Burkhart, S., Knezevic, M., Donaubauer, A., Muhammad, A., Asseng, S., Kolbe, T. H., & Noack, P. (2026). *Enabling high-throughput crop growth simulation with SciWIn*. Zenodo. https://doi.org/10.5281/zenodo.21099598

Gustafsson, O. J. R., Wilkinson, S. R., Bacall, F., Soiland-Reyes, S., Leo, S., Pireddu, L., Owen, S., Juty, N., Fernández, J. M., Brown, T., Ménager, H., Grüning, B., Capella-Gutierrez, S., Coppens, F., & Goble, C. (2025). WorkflowHub: A registry for computational workflows. *Scientific Data*, *12*(1), 837. https://doi.org/10.1038/s41597-025-04786-3

Kanitz, A., McLoughlin, M. H., Beckman, L., Malladi, V. S., & Ellrott, K. (2024). The GA4GH task execution application programming interface: Enabling easy multicloud task execution. *Computing in Science & Engineering*, *26*(3), 30–39. https://doi.org/10.1109/MCSE.2024.3414994

Kaushik, G., Ivkovic, S., Simonovic, J., Tijanic, N., Davis-Dusenbery, B., & Kural, D. (2016). RABIX: AN OPEN-SOURCE WORKFLOW EXECUTOR SUPPORTING RECOMPUTABILITY AND INTEROPERABILITY OF WORKFLOW DESCRIPTIONS.

DRAFT

Biocomputing 2017, *22*, 154–165. https://doi.org/10.1142/9789813207813{\textunderscore}0016

Krumsieck, J. (2025). *SciWIn – making your workflows work for you*. https://fairagro.net/event/fairagro-talk-sciwin-making-your-workflows-work-for-you/

Krumsieck, J., Leidel, A., König, P., & Waldow, H. von. (2025a). *Easy creation of reproducible computational workflows with SciWIn-client*. Zenodo. https://doi.org/10.5281/zenodo.16735806

Krumsieck, J., Leidel, A., König, P., & Waldow, H. von. (2025b). *Easy creation of reproducible computational workflows with SciWIn-client*. Zenodo. https://doi.org/10.5281/zenodo.17119086

Krumsieck, J., Leidel, A., Stiensmeier, X., & Waldow, H. von. (2025). *FAIR, fast, and frictionless – computational workflows with SciWIn*. Zenodo. https://doi.org/10.5281/zenodo.17651648

Krumsieck, J., Leidel, A., Stiensmeier, X., & Waldow, H. von. (2026a). *SciWIn-client and SciWIn-studio: Simplifying FAIR computational workflows*. Zenodo. https://doi.org/10.5281/zenodo.19060805

Krumsieck, J., Leidel, A., Stiensmeier, X., & Waldow, H. von. (2026b). *SciWIn – enabling FAIR for scientific analysis workflows*. Zenodo. https://doi.org/10.5281/zenodo.23154003

Leidel, A., Stiensmeier, X., Krumsieck, J., & Waldow, H. von. (2026). *Reproducible CWL workflows made easy with SciWIn*. Zenodo. https://doi.org/10.5281/zenodo.23156344

Leo, S., Crusoe, M. R., Rodríguez-Navas, L., Sirvent, R., Kanitz, A., Geest, P. de, Wittner, R., Pireddu, L., Garijo, D., Fernández, J. M., Colonnelli, I., Gallo, M., Ohta, T., Suetake, H., Capella-Gutierrez, S., Wit, R. de, Kinoshita, B. P., & Soiland-Reyes, S. (2024). Recording provenance of workflow runs with RO-crate. *1932-6203*, *19*(9), e0309210. https://doi.org/10.1371/journal.pone.0309210

Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., van Dyken, P. C., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Vieira, F. G., Meesters, C., Lee, S., Twardziok, S. O., Kanitz, A., VanCampen, J., Malladi, V., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., & Köster, J. (2021). Sustainable data analysis with snakemake. *F1000Research*, *10*, 33. https://doi.org/10.12688/f1000research.29032.3

Piccolo, S. R., Ence, Z. E., Anderson, E. C., Chang, J. T., & Bild, A. H. (2021). Simplifying the development of portable, scalable, and reproducible workflows. *eLife*, *10*. https://doi.org/10.7554/eLife.71069

Pimentel, J. F., Murta, L., Braganholo, V., & Freire, J. (2017). noWorkflow. *Proceedings of the VLDB Endowment*, *10*(12), 1841–1844. https://doi.org/10.14778/3137765.3137789

PMCC Bioinformatics Core. (2023). *Janis*. GitHub. https://github.com/PMCC-BioinformaticsCore/janis

Rabix Composer contributors. (2021). *Rabix composer*. https://github.com/rabix/composer

Rasche, H., & Khodak, A. (2024). *argparse2tool: Transparently build CWL and Galaxy XML tool definitions for any script that uses argparse*. GitHub. https://github.com/hexylena/argparse2tool

Šimko, T., Heinrich, L., Hirvonsalo, H., Kousidis, D., & Rodríguez, D. (2019). REANA: A system for reusable research data analyses. *2100-014X*, *214*, 06034. https://doi.org/10.1051/epjconf/201921406034

Soiland-Reyes, S., Sefton, P., Crosas, M., Castro, L. J., Coppens, F., Fernández, J. M., Garijo, D., Grüning, B., La Rosa, M., Leo, S., Ó Carragáin, E., Portier, M., Trisovic, A., Community, R.-C., Groth, P., & Goble, C. (2022). Packaging research artefacts with

DRAFT

RO-crate. *24518492*, *5*(2), 97–138. https://doi.org/10.3233/DS-210053

Soranzo, N. (2021). *Pull-request: Implement a subset of the common workflow language*. https://github.com/galaxyproject/galaxy/pull/12909

Tanjo, T., & Ishii, M. (2022). *Zatsu-cwl-generator: Simple CWL document generator from given execution commands*. GitHub. https://github.com/tom-tan/zatsu-cwl-generator

The Galaxy Community. (2024). The galaxy platform for accessible, reproducible, and collaborative data analyses: 2024 update. *Nucleic Acids Research*, *52*(W1), W83–W94. https://doi.org/10.1093/nar/gkae410

Ton That, D. H., Fils, G., Yuan, Z., & Malik, T. (2017). Sciunits: Reusable research objects. *2017 IEEE 13th International Conference on e-Science (e-Science)*, 374–383. https://doi.org/10.1109/eScience.2017.51

Vivian, J., Rao, A. A., Nothaft, F. A., Ketchum, C., Armstrong, J., Novak, A., Pfeil, J., Narkizian, J., Deran, A. D., Musselman-Brown, A., Schmidt, H., Amstutz, P., Craft, B., Goldman, M., Rosenbloom, K., Cline, M., O'Connor, B., Hanna, M., Birger, C., … Paten, B. (2017). Toil enables reproducible, open source, big biomedical data analyses. *Nature Biotechnology*, *35*(4), 314–316. https://doi.org/10.1038/nbt.3772

Weil, H. L., Schneider, K., Tschöpe, M., Bauer, J., Maus, O., Frey, K., Brilhaus, D., Martins Rodrigues, C., Doniparthi, G., Wetzels, F., Lukasczyk, J., Kranz, A., Grüning, B., Zimmer, D., Deßloch, S., Suchodoletz, D. von, Usadel, B., Garth, C., & Mühlhaus, T. (2023). PLANTdataHUB: A collaborative platform for continuous FAIR data sharing in plant research. *1365-313X*, *116*(4), 974–988. https://doi.org/10.1111/tpj.16474

Zwaan, J. M. van der, Andela, B., Vries, B. de, Veen, L., Haren, R. van, Verhoeven, S., & Ploeg, A. van der. (2019). *Scriptcwl* (Version 0.8.1). Zenodo. https://doi.org/10.5281/zenodo.3365635

DRAFT